

Labo-opdracht 3: I²C

Benodigdheden:

- Nucleo-L476RG met mini USB-kabel
- MPU-9250
- Jumperkabels: 4x F/F
- STM32CubeMX en Keil μ Vision
- Codevoorbeeld *I2C - polling - MPU9250 met magnetometer - template*

Los onderstaande vragen op in het sjabloon in hetzelfde project.

In het sjabloon is er her en der TODO te vinden. Die aanduidingen kunnen je helpen om de code vlotter op te stellen.

Let op: het *main.c*-bestand mag niet gewijzigd worden! Het moet onaangeroerd blijven en er dus steeds blijven uitzien zoals hieronder. Je kan eventueel wel tijdelijk een aantal statements in commentaar zetten tijdens het testen.

```
117 // Who am I opvragen via polling.
118 printf("MPU9250 who am I: %d.\r\n", WhoAmIMpu9250(&hi2c1));
119
120 // Magnetometer device ID opvragen via polling.
121 printf("Magnetometer device ID: %d.\r\n", ReadIdAK8963(&hi2c1));
122
123 /* USER CODE END 2 */
124
125 /* Infinite loop */
126 /* USER CODE BEGIN WHILE */
127 while (1)
128 {
129     // Accelerometer, gyroscoop en magnetometer uitlezen via polling.
130     sensordata = ReadMpu9250(&hi2c1);
131
132     // JSON dumpen op UART2.
133     printf("{ \"accx\":%d, \"accy\":%d, \"accz\":%d, \"gyrox\":%d, \"gyroy\":%d, \"gyroz\":%d, \"\n"
134            " \"magnx\":%d, \"magny\":%d, \"magnz\":%d}\r\n",
135            sensordata.accX, sensordata.accY, sensordata.accZ, sensordata.gyroX, sensordata.gyroY,
136            sensordata.gyroZ, sensordata.magnX, sensordata.magnY, sensordata.magnZ);
137
138     // Teken van leven geven.
139     ToggleLed(8);
140
141     // Even wachten. Uitlezen van de I2C-bus duurt ongeveer 2,5ms (2 + 14 + 2 + 7 bytes).
142     HAL_Delay(20);
```

Je zal ook allerhande datasheets van de MPU-9250 en de HAL-bibliotheken nodig hebben om info op te zoeken.

Uiteraard moet je ook zo'n MPU-9250 sensor koppelen via jumper kabels met je Nucleo-bord. Let erop dat je de massa (GND) en voedingsspanning (VCC) correct verbindt. De voedingsspanning mag niet hoger zijn dan 3,3V!

Labo-opdracht 3.1: MPU-9250 'Who am I' via polling

Vul zelf de code aan om de functie `WhoAmIMpu9250()` tot leven te wekken. Wijzig de signatuur van de functie niet! Gebruik enkel blokkerende HAL-functies tijdens de I²C-communicatie.

De 'Who am I' data die je kan verwachten, is terug te vinden in de datasheet(s) van de sensor.

Labo-opdracht 3.2: magnetometer 'device ID' via polling

Vul zelf de code aan om de functie `ReadIdAK8963()` tot leven te wekken. Wijzig de signatuur van de functie niet! Gebruik enkel blokkerende HAL-functies tijdens de I²C-communicatie.

De AK8963 verwijst naar een tweede ingebouwde sensor in de MPU-9250 chip. Die AK8963 is een magnetometer. Daarmee kunnen we magnetische velden opzoeken en is het dus mogelijk om een kompas te maken.

De *device ID* data die je kan verwachten, is terug te vinden in de datasheet(s) van de sensor.

Labo-opdracht 3.3: MPU-9250 gyroscoop, accelerometer en magnetometer via polling

Bedenk zelf de (enige) correcte signatuur voor de `ReadMpu9250()` functie. Stel die op in C-code en vervul de functie zodat ze na afloop al de sensordata in een `Mpu9250`-variabele bevat (9 assen). Gebruik dus ook hier weer blokkerende HAL-functies.

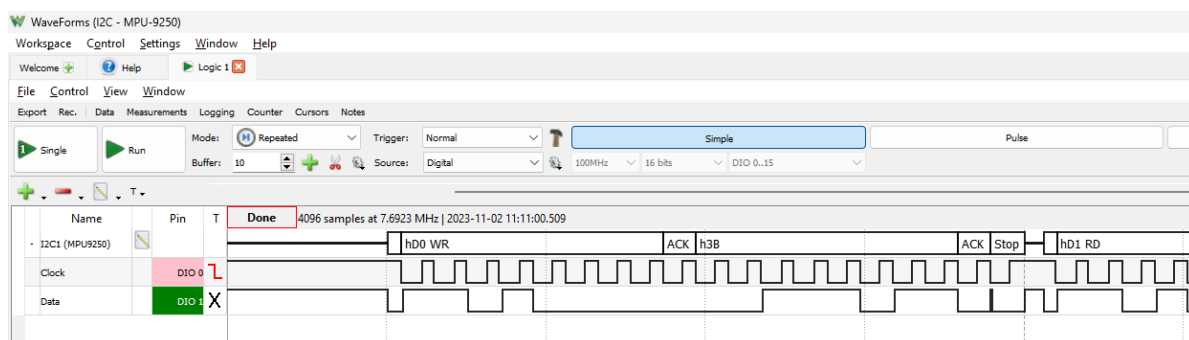
Let op: om de informatie van de magnetometer uit te lezen, volstaat het niet om enkel de 6 bytes met *measurement data* uit te lezen. Je moet ook het ST2 register lezen om de sensor (AK8963) te laten weten dat je alle *measurement data* effectief uitlas.

Klaar met je aanpassingen? Bewijs dat alles werkt door de ongewijzigde hoofdlus uit te voeren in je Nucleo-bord en de JSON te bekijken in bijvoorbeeld Putty.

Labo-opdracht 3.4: I²C-communicatie visualiseren

Als kers op de taart, kan je de I²C-communicatie gaan visualiseren. Gebruik daarvoor een oscilloscoop.

Je zou een beeld moeten bekomen zoals hieronder. Uiteraard met nog meer data in beeld. Maak hiervan een screenshot en plaats deze ook in de repo.



In te dienen via repo in de **main-branch**:

- Een map met het STM32CubeMX/Keil-project

Bron: R. Buyschaert